

京东前端面试-大模型方向

一面（技术基础）

1. 简单介绍一下 JavaScript 的数据类型，以及如何准确判断一个变量是否为数组？

难度：★

考点：JS 基础类型、类型检测

答案要点：

JavaScript 分为基本数据类型和引用数据类型，判断数组最稳妥的方法是使用原生提供的静态方法。

- 基本类型：String, Number, Boolean, Null, Undefined, Symbol, BigInt
- 引用类型：Object, Array, Function, Date, RegExp
- 判断数组：Array.isArray() 是首选，也可以使用 Object.prototype.toString.call()
- 区别：typeof 对数组会返回 object，instanceof 在多窗口/iframe 环境下可能失效

常见坑：

- 认为 typeof 可以区分数组和对象
- 忽略了 null 的 typeof 结果是 object

追问：

- 为什么 typeof null 是 object?
- 如何判断一个对象是否为空对象？

2. 说一下 Promise 的状态有哪些，以及 Promise.all 和 Promise.allSettled 的区别？

难度：★

考点：异步编程、Promise API

答案要点：

Promise 有三种状态，all 和 allSettled 主要区别在于对失败状态的处理逻辑不同。

- 三种状态：Pending（进行中）、Fulfilled（已成功）、Rejected（已失败）
- Promise.all：要求所有任务都成功才返回成功，只要有一个失败就立即进入 reject
- Promise.allSettled：无论成功还是失败，都会等所有任务结束，并返回每个任务的结果数组
- 场景：all 适用于强依赖的一组请求，allSettled 适用于互不影响的并行任务

常见坑：

- 忘记 Promise 状态一旦改变就不可逆
- 在 allSettled 的结果中忘记区分 status 字段

追问：

- 如果 Promise.all 中某个请求失败了，如何保证其他成功的请求结果不被丢弃？

3. 在 CSS 布局中，Flex 布局的 flex: 1 代表什么含义？

难度：★

考点：CSS 布局、Flexbox 属性

答案要点：

flex: 1 是一个简写属性，它决定了子元素如何分配父容器的剩余空间。

- 组成部分：它是 flex-grow: 1, flex-shrink: 1, flex-basis: 0% 的缩写
- flex-grow：定义放大比例，1 表示占用剩余空间的 1 份
- flex-shrink：定义缩小比例，1 表示空间不足时等比例缩小
- flex-basis：定义在分配多余空间之前的项目占据的主轴空间

常见坑：

- 误以为 flex: 1 等同于 flex: 1 1 auto（实际 basis 是 0%）
- 不清楚 flex-basis 与 width 的优先级关系

追问：

- flex: auto 和 flex: none 分别代表什么？

4. 谈谈你对浏览器事件循环（Event Loop）的理解，微任务和宏任务有哪些？

难度：★★

考点：浏览器原理、执行机制

答案要点：

事件循环是浏览器协调 JS 单线程执行与异步任务的机制，通过任务队列实现非阻塞 IO。

- 执行顺序：先执行同步代码（调用栈），再清空微任务队列，最后取出一个宏任务执行
- 微任务：Promise.then、MutationObserver、queueMicrotask
- 宏任务：script 整体代码、setTimeout、setInterval、setImmediate、I/O
- 渲染时机：浏览器通常在微任务队列清空后、下一个宏任务开始前尝试进行 UI 渲染

常见坑：

- 认为 async/await 是宏任务（await 后的代码会被包装成微任务）
- 混淆了 Node.js 和浏览器的事件循环差异

追问：

- 如果在微任务里不停地递归产生新的微任务，会发生什么？

5. TypeScript 中的 interface 和 type 有什么区别？

难度：★

考点：TS 基础语法

答案要点：

两者都可以定义对象结构，但 interface 侧重于结构的声明和合并，type 侧重于类型的定义和组合。

- 扩展方式：interface 使用 extends，type 使用交叉类型 &
- 声明合并：同名 interface 会自动合并，同名 type 会报错
- 能力差异：type 可以定义基本类型别名、联合类型、元组，interface 不行
- 适用场景：定义公共 API 建议用 interface，定义复杂逻辑或联合类型用 type

常见坑：

- 试图在 type 中使用 implements 关键字
- 忽略了 interface 无法描述非对象类型

追问：

- 什么是泛型？在 API 接口请求中如何使用泛型约束返回结果？

6. 什么是 SSE (Server-Sent Events)？它与 WebSocket 有什么区别？

难度：★★

考点：网络协议、流式传输

答案要点：

SSE 是一种基于 HTTP 的单向推送技术，非常适合大模型这种流式输出文本的场景。

- 协议特性：基于标准 HTTP 协议，支持断线重连，数据格式默认为 text/event-stream
- 方向性：SSE 是单向的（服务端到客户端），WebSocket 是全双工的（双向）
- 复杂度：SSE 实现简单，对代理服务器友好；WebSocket 协议较重，需要独立握手
- AI 应用：大模型生成回复时，前端通常通过 SSE 接收逐字返回的结果

常见坑：

- 认为 SSE 可以发送二进制数据（原生仅支持文本）
- 忽略了浏览器对同一域名下 SSE 连接数的限制（HTTP/1.1 下通常为 6 个）

追问：

- 如果后端接口是 POST 请求，还能用 SSE 吗？（提示：fetch 的 ReadableStream）

7. 如何实现一个深拷贝函数？需要考虑哪些特殊情况？

难度：★★

考点：手写代码、递归、边界处理

答案要点：

深拷贝需要递归遍历对象，并处理循环引用以及特殊内置对象。

- 基础实现：使用递归，判断类型是否为对象
- 循环引用：使用 WeakMap 存储已拷贝的对象，防止死循环
- 特殊类型：处理 Date、RegExp、Map、Set 等特殊对象的克隆
- 性能考虑：避免对超大对象进行深拷贝，或者使用 structuredClone API

常见坑：

- 使用 JSON.parse(JSON.stringify()) 导致函数、正则、Date 丢失或变形
- 递归未设置终止条件导致栈溢出

追问：

- 为什么处理循环引用要用 WeakMap 而不是 Map？

8. 写一个简单的防抖（debounce）函数。

难度：★

考点：手写代码、闭包、高阶函数

答案要点：

防抖是指在事件触发后延迟执行，如果在延迟时间内再次触发，则重新计时。

- 核心逻辑：利用闭包维护一个 timer 变量
- 关键步骤：返回一个新函数，每次调用先 clearTimeout，再重新 setTimeout
- 应用场景：搜索框输入联想、窗口大小调整（resize）
- 优化：支持立即执行参数（leading）

代码示例：

代码块

```
1 function debounce(fn, delay) {
2   let timer = null;
3   return function(...args) {
4     if (timer) clearTimeout(timer);
5     timer = setTimeout(() => {
6       fn.apply(this, args);
7     }, delay);
8   };
9 }
```

常见坑：

- 丢失了原函数的 this 指向
- 忘记传递 arguments 参数

9. 算法题：给定一个字符串，找出其中不含有重复字符的最长子串的长度。

难度：★★

考点：滑动窗口、字符串处理

答案要点：

使用滑动窗口的思想，通过两个指针维护一个不含重复字符的区间。

- 核心思路：左指针不动，右指针向右移动，并用 Map 记录字符出现的位置
- 遇到重复：如果右指针指向的字符已存在，则将左指针移动到重复字符位置的下一个
- 更新结果：每次移动右指针都计算当前窗口长度，取最大值
- 复杂度：时间复杂度 $O(n)$ ，空间复杂度 $O(\min(m, n))$

常见坑：

- 左指针回退问题（必须保证左指针只能向右跳）
- 字符串为空或长度为 1 的边界处理

10. 在 Web 开发中，如何实现一个简单的打字机效果？

难度：★★

考点：DOM 操作、定时器、AI 交互

答案要点：

打字机效果可以通过定时器逐个追加字符，或者在 AI 场景下根据流式数据动态更新。

- 实现方式：使用 `setInterval` 或 `requestAnimationFrame` 配合 `slice` 截取字符串
- 样式配合：利用 CSS 伪元素实现光标闪烁效果
- AI 场景：直接将 SSE 接收到的 chunk 累加到响应式变量中，由框架驱动视图更新
- 性能优化：对于长文本，避免频繁操作 DOM，可以使用虚拟 DOM 批量更新

常见坑：

- 文本包含 HTML 标签时，简单的 `slice` 会导致标签被截断引起页面乱码
- 忘记在组件销毁时清除定时器

追问：

- 如果文本中包含 Markdown 语法，如何保证打字机效果下渲染正确？

二面（深入原理与项目）

1. React 的 Fiber 架构解决了什么问题？它的工作原理是怎样的？

难度：★★★★

考点：React 核心原理、调度机制

答案要点：

Fiber 主要是为了解决 React 在处理大量 DOM 更新时导致的页面卡顿问题，将同步更新变为可中断的异步更新。

- 核心矛盾：JS 引擎与 GUI 渲染互斥，长耗时任务会导致丢帧
- 解决手段：引入时间切片（Time Slicing），将更新过程拆分为多个小任务
- 双缓存技术：在内存中构建 WorkInProgress Fiber 树，完成后一次性提交给真实 DOM
- 优先级调度：根据任务紧急程度（如用户输入 vs 数据请求）分配不同的优先级

常见坑：

- 认为 Fiber 提高了代码执行速度（实际上增加了计算量，但提升了响应性）
- 混淆了 reconciliation（协调）和 commit（提交）阶段的可中断性

追问：

- 为什么 Hooks 必须在顶层调用，不能写在 if 语句里？

2. 在大模型对话应用中，前端如何处理长文本的流式渲染以保证不卡顿？

难度：★★★

考点：性能优化、流式数据处理、大模型应用

答案要点：

长文本渲染的瓶颈在于频繁的 DOM 树重构和 Markdown 解析开销，需要从解析策略和渲染频率入手。

- 节流渲染：不要每收到一个 token 就渲染一次，可以按时间间隔（如 50ms）批量更新
- 局部解析：使用支持流式解析的 Markdown 库，或者只对新增部分进行增量解析
- 虚拟列表：如果对话历史非常长，必须引入虚拟滚动（Virtual List）只渲染可视区
- 样式优化：避免在文本追加时触发大规模的重排（Reflow），固定容器宽高或使用 contain 属性

常见坑：

- 每次更新都全量解析整个对话字符串
- 忽略了自动滚动到底部逻辑在高频更新下的性能损耗

追问：

- 如何实现对话过程中的“自动滚动到底部”且允许用户手动向上滚动查看历史？

3. 谈谈你对 Prompt Engineering（提示工程）在前端开发中应用的理解。

难度：★★

考点：AI 应用、前端工程化

答案要点：

前端在 Prompt 工程中承担着模板管理、上下文构建和输入约束的重要角色。

- 模板管理：将通用的 Prompt 抽离为配置化模板，支持变量注入（如用户上下文、历史记录）
- 结构化输出：通过 Prompt 强制模型返回 JSON 格式，方便前端逻辑解析和 UI 渲染

- 动态上下文：根据用户当前的操作路径，自动在后台拼接相关的系统提示词（System Prompt）
- 提示词安全：在前端进行敏感词过滤，防止用户通过注入攻击绕过模型限制

常见坑：

- 直接在前端代码里硬编码长篇累牍的 Prompt 字符串
- 忽略了 Token 长度限制，没有对上下文进行合理的裁剪

追问：

- 如果模型返回的 JSON 格式不完整（流式传输中），前端该如何处理解析错误？

4. 什么是 RAG（检索增强生成）？前端在 RAG 流程中可以做什么工作？

难度：★★★

考点：RAG 架构、AI 业务场景

答案要点：

RAG 通过检索外部知识库来弥补模型训练数据的时效性问题，前端主要负责交互展示和元数据反馈。

- 引用溯源：展示模型回答所依据的文档片段，并支持点击跳转到原文档具体位置
- 检索反馈：允许用户对检索到的内容进行“准确/不准确”的点赞踩，用于优化 Embedding 质量
- 文档处理：在上传知识库阶段，前端可以进行文件预处理、分段预览以及元数据编辑
- 状态可视化：展示检索中、思考中、生成中等细分状态，缓解用户等待焦虑

常见坑：

- 溯源链接失效或定位不准
- 忽略了检索到的敏感信息在前端的脱敏处理

追问：

- 如何在前端实现 PDF 文档的关键词高亮定位？

5. 如何设计一个通用的 AI Chat 组件？需要暴露哪些 Props 和 Slots？

难度：★★

考点：组件设计、抽象能力

答案要点：

一个好的 AI 组件应该具备高度的可定制性，同时封装复杂的流式交互逻辑。

- 核心 Props：messages（消息列表）、loading（生成状态）、onSend（发送回调）、renderMessage（自定义渲染）
- Slots 设计：inputArea（输入框区域）、avatar（头像）、actionButtons（操作按钮如重新生成）
- 内部状态：管理打字机效果、滚动位置、错误处理、复制功能
- 扩展性：支持多模态输入（图片、文件）、支持插件工具（Tool Call）的 UI 展示

常见坑：

- 组件逻辑太重，与特定的 API 强耦合
- 忽略了移动端适配和键盘弹起时的布局问题

追问：

- 如果要支持代码块的高亮和一键复制，你会怎么集成？

6. Web Worker 在 AI 相关的 Web 应用中有什么使用场景？

难度：★★

考点：多线程、前端计算性能

答案要点：

Web Worker 可以将耗时计算移出主线程，防止 AI 应用在处理大量数据时界面假死。

- 本地模型运行：如果使用 WebLLM 或 Transformers.js 在浏览器运行小模型，必须在 Worker 中执行
- 向量计算：在前端进行简单的向量相似度计算（如本地搜索）
- 数据预处理：对上传的大型文档进行分段（Chunking）和清洗
- 复杂解析：解析超大型的 JSON 或 Markdown 结果

常见坑：

- 在 Worker 中尝试操作 DOM
- 频繁在主线程和 Worker 间传递超大对象（应使用 Transferable Objects）

追问：

- 什么是 SharedWorker？它和普通 Worker 有什么区别？

7. 你们项目里是如何处理大模型调用超时或失败的？

难度：★★

考点：容错处理、用户体验

答案要点：

AI 接口通常响应较慢且不稳定，需要建立完善的重试、降级和反馈机制。

- 自动重试：针对网络波动或 5xx 错误，实现指数退避算法的自动重试
- 熔断机制：如果模型持续报错，切换到备用模型（如从 GPT-4 降级到 GPT-3.5）
- 分段超时：不仅设置总超时，还要设置首包超时（First Token Timeout）
- 交互反馈：明确告知用户错误原因（如 Token 超限、内容违规），并提供“重新生成”按钮

常见坑：

- 超时时间设置过短，导致长文本生成被误杀
- 报错后没有清理当前的流连接，导致内存泄漏

追问：

- 如何在前端计算当前对话消耗的 Token 数量？

8. 谈谈微前端架构（如 qiankun）的原理，以及在大型 AI 门户中的应用价值。

难度：★★★

考点：微前端、架构设计

答案要点：

微前端通过将巨石应用拆分为多个独立子应用，解决了跨团队协作和技术栈无关的问题。

- 核心原理：基于 single-spa，通过 HTML Entry 加载子应用，利用 JS 沙箱和 CSS 隔离保证互不干扰
- 应用价值：AI 门户通常包含对话、知识库、Agent 编排等多个复杂模块，适合独立开发部署
- 资源共享：通过主应用下发 AI 能力（如统一的 Token 校验、模型调用 SDK）
- 性能优化：支持子应用预加载，提升模块切换速度

常见坑：

- JS 沙箱导致的全局变量丢失问题
- 子应用间的样式污染（特别是弹窗组件）

追问：

- 微前端环境下，多个子应用如何共享同一个大模型连接的状态？

9. 什么是 TypeScript 的装饰器（Decorators）？在 AI 框架中有什么潜在用途？

难度：★★

考点：TS 高级特性

答案要点：

装饰器是一种特殊类型的声明，可以附加到类、方法、属性上，起到元编程的作用。

- 语法本质：是一个高阶函数，在类定义时执行
- 潜在用途：在 AI SDK 中，可以用装饰器来标记某个方法为“Tool/Function Call”，自动提取参数 Schema
- 日志监控：通过装饰器自动记录每个模型接口的输入输出和耗时
- 权限控制：校验当前用户是否有权调用特定的高阶模型

常见坑：

- 依赖 experimentalDecorators 配置，且旧版与新版标准有差异
- 装饰器会增加代码的理解成本和调试难度

追问：

- 装饰器是在运行时执行还是编译时执行？

10. 如何优化前端静态资源的加载速度？结合京东的业务场景谈谈。

难度：★★

考点：性能优化、工程化

答案要点：

优化应从减少请求体积、增加并发、利用缓存三个维度展开。

- 资源压缩：Gzip/Brotli 压缩、图片 WebP 化、代码混淆与 Tree Shaking
- 缓存策略：强缓存（Cache-Control）配合文件 Hash、协商缓存（Etag）
- 分包策略：利用 Vite/Webpack 的 SplitChunks 拆分第三方库，按需加载路由
- 传输优化：HTTP/2 多路复用、CDN 加速、预加载（prefetch/preload）

常见坑：

- Hash 改变导致缓存大面积失效
- 忽略了第三方 SDK（如监控、统计脚本）对首屏的影响

追问：

- 什么是 FCP 和 LCP？如何监控这些指标？

11. 手写题：实现一个带并发限制的异步调度器 Scheduler。

难度：★★★

考点：异步控制、队列思想

答案要点：

需要维护一个执行队列和当前正在运行的任务计数器。

- 核心逻辑：add 方法返回 Promise，并将任务包装后放入队列
- 触发执行：每当有任务完成，计数器减一，并从队列中取新任务执行
- 状态管理：确保并发数永远不超过设定的 limit

代码示例：

代码块

```
1  class Scheduler {
2    constructor(limit) {
3      this.limit = limit;
4      this.running = 0;
5      this.queue = [];
6    }
7    add(fn) {
8      return new Promise(resolve => {
9        const task = () => fn().then(resolve).finally(() => {
10          this.running--;
11          this.next();
12        });
13        this.queue.push(task);
14        this.next();
15      });
16    }
17  }
```

```
17     next() {
18         while (this.running < this.limit && this.queue.length) {
19             this.running++;
20             const task = this.queue.shift();
21             task();
22         }
23     }
24 }
```

12. 谈谈对 Web 安全的理解，特别是针对 AI 应用可能存在的安全风险。

难度：★★

考点：Web 安全、AI 安全

答案要点：

除了传统的 XSS/CSRF，AI 应用面临独特的提示词注入和敏感数据泄露风险。

- 传统安全：防范 XSS（对模型输出的 HTML 进行转义）、CSRF（使用 Token）
- 提示词注入：用户通过特定输入诱导模型跳出安全限制，前端应做初步过滤
- 数据脱敏：在将用户数据发送给外部模型 API 前，识别并遮蔽手机号、密钥等敏感信息
- 内容安全：对模型生成的图片或文字进行合规性校验（通常配合后端审核接口）

常见坑：

- 盲目信任模型返回的字符串，直接用 v-html 或 dangerouslySetInnerHTML 渲染
- 在前端代码或环境变量中暴露模型供应商的 API Key

追问：

- 如何防范点击劫持（Clickjacking）？

三面（架构与综合能力）

1. 如果让你从零搭建一个企业级 AI Agent 编排平台的前端架构，你会如何选型和设计？

难度：★★★

考点：架构设计、系统思考

答案要点：

架构设计应遵循模块化、可扩展和高性能的原则，重点处理复杂的交互逻辑。

- 技术选型：React + TS + TailwindCSS（快速 UI 开发）、Zustand/Redux（状态管理）、React Flow（流程图画布）
- 核心模块：可视化画布（节点编排）、资产中心（Prompt/Tool 管理）、调试控制台（日志与 Trace）、插件系统

- 数据流设计：采用单向数据流，画布操作实时同步至状态机，支持撤销重做（Undo/Redo）
- 性能保障：画布节点过多时采用 Canvas 渲染或分层渲染，大模型输出采用 SSE 独立通道

常见坑：

- 逻辑全部耦合在画布组件中，导致难以维护
- 忽略了不同 Agent 节点间的数据 Schema 校验

追问：

- 如何解决流程图中循环依赖的问题？

2. 在 AI 项目中，你是如何定义和衡量“前端用户体验”的？有哪些具体的指标？

难度：★★

考点：质量保障、指标体系

答案要点：

AI 应用的体验不仅看加载速度，更看交互的流畅度和反馈的及时性。

- 核心指标：First Token Latency（首字响应时间）、Generation Speed（生成速度 chars/s）、Request Success Rate
- 交互指标：输入框响应延迟、长文本渲染是否掉帧、自动滚动触发的准确率
- 业务指标：用户点赞/点踩比率、对话完成率、Agent 任务执行成功率
- 监控手段：接入全链路埋点，记录模型返回的 trace_id 方便排查前端渲染异常

常见坑：

- 只关注首屏加载时间，忽略了模型生成过程中的卡顿
- 缺乏对模型报错后的用户行为监控

3. 谈谈你在过去一年中遇到的最具有挑战性的技术问题，你是如何解决的？

难度：★★★

考点：解决问题能力、技术深度

答案要点：

这是一个开放性问题，建议按 STAR 原则（情境、任务、行动、结果）组织回答。

- 描述背景：问题发生的场景（如：高并发下的流式渲染崩溃、复杂的跨端兼容性问题）
- 分析过程：使用了哪些工具（Chrome DevTools, Performance tab, Wireshark）定位根因
- 解决方案：对比了哪几种方案，为什么最终选择了这一种（权衡利弊）
- 最终成果：性能提升了多少、错误率降低了多少、或者沉淀了什么通用组件库

常见坑：

- 描述得太泛，没有具体的技术细节
- 解决方案过于简单，体现不出“挑战性”

4. 面对 AI 技术的快速迭代（如从单模态到多模态），前端开发者应该如何保持竞争力？

难度：★★

考点：技术视野、学习能力

答案要点：

前端开发者应从“页面实现者”转型为“AI 能力的整合者和交互设计者”。

- 深度掌握 AI 基础设施：熟悉 LangChain.js、Vercel AI SDK 等主流 AI 开发框架
- 探索新交互形态：研究语音交互（VAD）、视觉输入、自然语言驱动的 UI（Generative UI）
- 强化工程化能力：学习如何处理海量向量数据、如何优化长上下文的性能瓶颈
- 关注底层原理：了解 Transformer 基本原理、Token 机制，以便更好地与算法同学沟通

常见坑：

- 焦虑于工具的更替，忽略了编程基础（JS/浏览器原理）的加固
- 只会调用 API，不理解 AI 业务背后的逻辑

5. 如何在团队中推动前端代码质量的提升？有哪些具体手段？

难度：★★

考点：团队管理、工程实践

答案要点：

提升代码质量需要从规范制定、工具约束和文化建设三个方面入手。

- 自动化工具：配置严苛的 ESLint/Prettier 规则，在 CI 阶段强制执行单元测试覆盖率检查
- Code Review：建立互审机制，关注逻辑健壮性、扩展性以及是否符合 AI 业务的最佳实践
- 文档建设：维护高质量的组件库文档和技术方案设计文档（RFC）
- 技术分享：定期进行 Case Study，分享踩坑经验，避免同类问题再次发生

常见坑：

- 规范只停留在纸面上，没有强制执行力
- CR 变成走过场，只看格式不看逻辑

6. 假设京东要上线一个“AI 购物助手”，你会如何设计它的首屏加载策略以应对大流量？

难度：★★★★

考点：高并发、性能架构

答案要点：

针对大流量场景，核心思路是“动静分离”和“多级降级”。

- 静态资源：利用 CDN 全球加速，采用多域名存储资源突破浏览器并发限制

- 预加载：在用户进入页面前，通过 Link Prefetch 预加载 AI 核心逻辑包
 - 接口优化：对话历史记录采用 SSR（服务端渲染）或边缘计算（Edge Computing）快速呈现
 - 柔性降级：流量过大时，关闭复杂的动画效果，甚至将流式输出切换为全量输出以减少连接数
- 常见坑：**
- 所有的 AI 逻辑都打包在一个大 JS 包里
 - 忽略了后端接口在高并发下的响应延迟对前端的影响

7. 你对“AI 驱动 UI 生成”（Generative UI）怎么看？它会取代前端吗？

难度：★★★

考点：行业洞察

答案要点：

Generative UI 是交互方式的进化，它改变了前端的生产模式，但不会取代前端。

- 核心观点：前端的工作将从“写死页面”转向“编写 UI 生成规则和约束”
- 价值所在：能够根据用户意图动态生成最合适的组件（如查机票返回机票卡片，查天气返回气象图）
- 挑战：如何保证生成 UI 的稳定性、可访问性（Accessibility）以及品牌一致性
- 结论：前端需要负责构建高质量的组件元数据（Storybook）和布局引擎，作为 AI 生成的“原材料”

常见坑：

- 观点过于极端（全盘肯定或全盘否定）
- 无法说出具体的应用场景

8. 聊聊你在项目管理或跨团队协作中，是如何处理需求冲突或进度延迟的？

难度：★★

考点：软技能、沟通协作

答案要点：

处理冲突的关键在于透明沟通和基于数据的决策。

- 明确优先级：使用四象限法则，优先保证核心链路（如 AI 对话主流程）的上线
- 风险预警：一旦发现进度可能延迟，第一时间同步给 PO 和相关方，并提供备选方案（如裁剪非核心功能）
- 技术复盘：分析延迟原因，是技术方案太复杂还是需求变更频繁，在下一代中改进
- 跨端协同：与后端/算法约定好接口契约（如 Protobuf 或 Swagger），减少联调耗时

常见坑：

- 闷头写代码，直到上线前才告知进度无法完成
- 抱怨其他部门，缺乏主动解决问题的态度